BENCHMARK · ALCALDÍA DE MEDELLÍN · RETO DE VALOR

OmegaHack vs. el proceso actual

Tres indicadores medidos sobre datos reales del tenant demo. Sin promesas: queries directas a Supabase, reproducibles por cualquier miembro del jurado.

Muestra	93 PQRs del tenant demo
Corrida	19 abr 2026 · 15:19 UTC
Script	`scripts/benchmark.ts` - `tsx scripts/benchmark.ts`
Datos crudos	`.private/benchmark/latest.json` (no versionado)

Resumen ejecutivo

#	INDICADOR	SIN OMEGAHACK	CON OMEGAHACK
M1	**Repetición** - dedup rate	0.0 %	**12.9 %** 8 respuestas ahorradas · 1 grupo hot
M2	**Cargas** - auto-routing	0.0 % triage manual	**81.7 %** 76 / 93 con secretaría · 82.8 % con prioridad
M3	**Tiempo** - intake p50	≈ 30 min triage humano (supuesto)	**10 min** n = 23 PQRs medidos

M1 · Repetición de casos

PROBLEMA

> *"El mismo caso puede llegar desde diferentes usuarios o plataformas, lo que genera una saturación en las solicitudes."*

CÓMO LO ATACA OMEGAHACK

El paquete `@omega/problem-groups` agrupa PQRs por similitud vectorial (cosine ≥ 0.80), tags compartidos (≥ 2) y comuna coincidente. Cada grupo se responde una sola vez en lugar de N.

FÓRMULA

- `dedup_rate = grouped_pqrs / total_pqrs` - miembros en `pqr_problem_group_members`.
- `respuestas_ahorradas = grouped_pqrs - active_groups` (`member_count ≥ 2`).
- `hot_groups` = `problem_groups.hot = true` (≥ 5 miembros en 5 días).

DEDUP RATE

12.9 %

12 / 93 PQRs agrupados

RESPUESTAS AHORRADAS

8

de 4 grupos activos

Total PQRs	**93**
Agrupados	**12**
Grupos activos (≥ 2)	**4**
Grupos *hot*	**1**

LECTURA

Con 12.9 % el sistema ya deduplica casi 1 de cada 8 casos y detecta un cluster *hot* que merece atención preferente. En un dataset más grande (miles de PQRs) la cobertura sube porque hay más probabilidad de encontrar un match sobre el umbral de similitud.

M2 · Cargas innecesarias

PROBLEMA

> *"Los funcionarios pierden tiempo respondiendo PQRS que no están bien 'organizados'. Mala asignación de casos según su enfoque."*

CÓMO LO ATACA OMEGAHACK

El intake-agent ejecuta el clasificador Claude (`@omega/classifier`), que asigna secretaría + prioridad + tags con confianza antes de que el caso llegue a la bandeja humana. El funcionario abre el PQR ya enrutado.

FÓRMULA

- `auto_routing_rate = pqrs_con_secretaria / total_pqrs` .
- `priority_rate = pqrs_con_priority_level / total_pqrs` .
- `high_confidence_rate = pqrs_con_tag_confidence≥0.7 / total_pqrs` .

AUTO-ROUTING

81.7 %

76 / 93 con secretaría

PRIORIDAD ASIGNADA

82.8 %

77 / 93 con P0-P3

TOP 5 SECRETARÍAS

55	Infraestructura Física
2	Seguridad y Convivencia
2	Educación
2	Gerencia del Centro
2	Movilidad

LECTURA

8 de cada 10 PQRs llegan ya enrutados a la secretaría correcta - el funcionario entra directo a redactar la respuesta en lugar de triar. La concentración en Infraestructura Física refleja el corpus actual (huecos, andenes, vías) y sugiere reforzar esa cola en la UI del workbench.

Nota. El indicador de tags de alta confianza quedó en 0 % porque las confidencias no están poblando el campo `pqr_tags.confidence` en el dataset actual (el clasificador marca tags pero no persiste score). Es un hueco de datos, no una falla del enrutamiento.

M3 · Tiempo de respuesta

PROBLEMA

> *"Para los funcionarios es importante cumplir con los tiempos de respuesta, pero por la falta de orden y otros factores, esta tarea se puede complicar."*

CÓMO LO ATACA OMEGAHACK

1. **Intake automatizado** - `@omega/intake-agent` valida, clasifica y prioriza en minutos.
2. **Deadline-engine** - plazo hábil (Ley 1755/2015 + Emiliani + suspensiones) y semáforo on_track / at_risk / overdue.

FÓRMULA

- **Intake latency**: `classified.ts - received.ts` (p50/p90/p99/max).
- **Salud de plazos**: bucket por `legal_deadline - now()`.

INTAKE LATENCY - PERCENTILES (N = 23)

p50	**10 min**
p90	10 min
p99	10 min
max	10 min

SEMÁFORO · 69 ABIERTOS

on_track (≥ 48 h margen)	60
at_risk (< 48 h margen)	5
overdue	4

END-TO-END · RECEIVED → RESPONSE_SENT (N = 3)

PERCENTIL	VALOR	≈ DÍAS
p50	216 h	~9
p90	238 h	~10
max	238 h	~10

LECTURA

El intake cumple su parte (10 min p50, consistente en toda la muestra). Los 9 casos en at_risk + overdue son exactamente los que el semáforo del workbench debe destacar al funcionario - hoy, sin OmegaHack, se detectan cuando ya es tarde.

Supuestos y limitaciones

- **Baseline "sin OmegaHack"** es un peor caso derivado del mismo dataset: dedup 0 %, auto-routing 0 %, triage humano ≈ 30 min por PQR. Supuesto conservador - no medido con cronómetro.
- Los eventos de `pqr_events` mezclan runs sintéticos (seed `synthetic-pqrs`) y reales. La latencia refleja el pipeline actual.

- La muestra (93 PQRs) es chica para percentiles altos: p99 coincide con p50. Se estabiliza al reingerir el corpus completo.
- El hueco `pqr_tags.confidence = NULL` distorsiona el indicador de tags - dato ausente, no routing malo.

Datos adicionales y siguiente paso

El archivo `PDS_Final_PQRSF.xlsx` (compartido por el equipo) contiene histórico PQRSF al que este benchmark aún no se conecta. Integrarlo permitirá:

- Baseline humano real - tiempos de respuesta pre-OmegaHack, medidos.
- Tamaño de muestra mayor para estabilizar percentiles.
- Validar la tasa de repetición con datos de producción, no sintéticos.

Reproducir

```
set -a && source .env && set +a
tsx scripts/benchmark.ts
```

Salida en consola y en `.private/benchmark/latest.{json,md}`.

Benchmark OmegaHack · corrida 2026-04-19 15:19 UTC · tenant `00000000…0001` · 93 PQRs. Script versionado en `scripts/benchmark.ts` - este documento refleja la salida literal; regenerar tras cada cambio en el corpus.